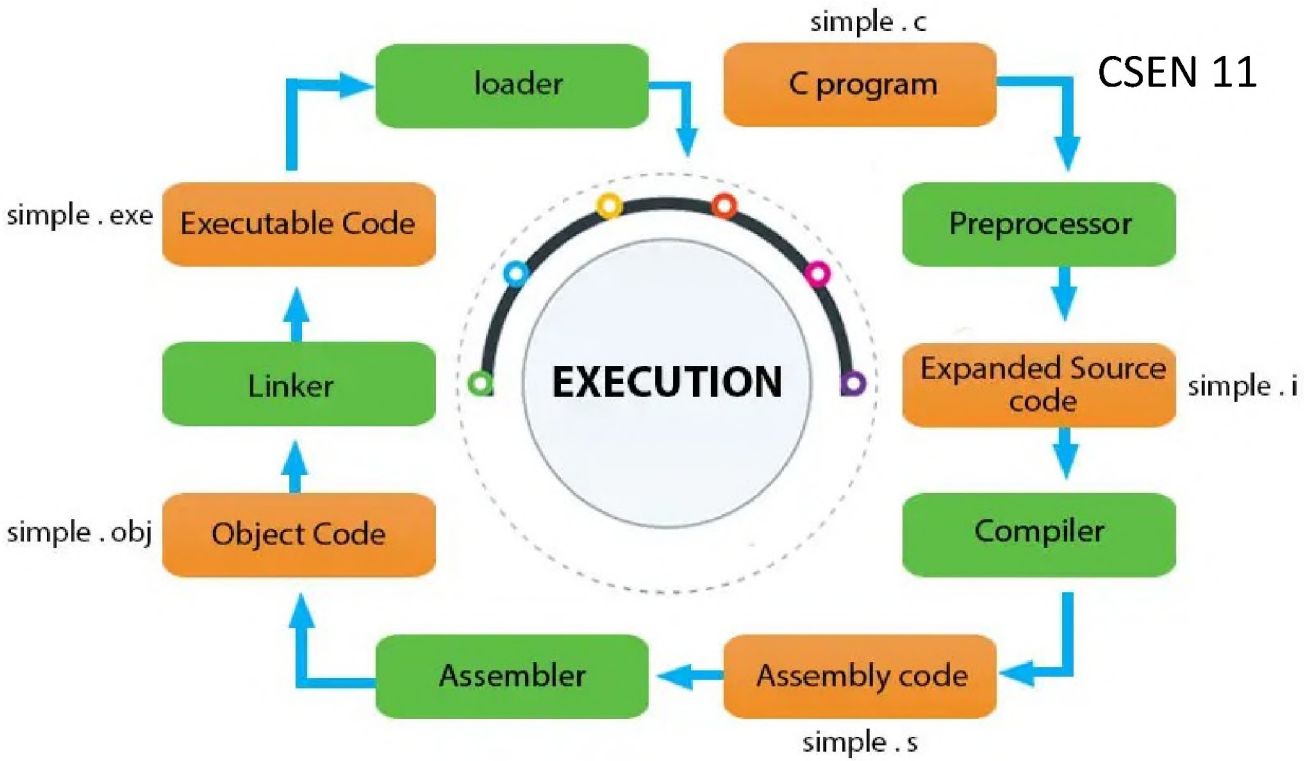
CSEN 11 Advanced Programming

Intro to C programming

C Program Lifecycle

This diagram shows the *entire life of a C program* — from the code you write to the program actually running on your computer.
You won't master all of this in one class. Each CS class peels off one layer.

These exist, but they're not the main focus of this course.



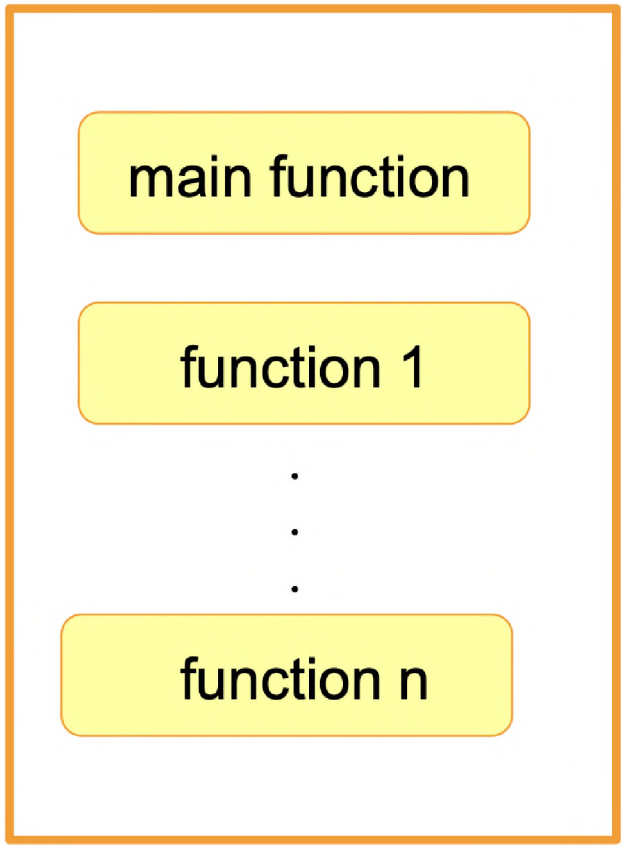
CSEN 11

You will learn how to think like a programmer and write correct C programs.

You won't build the preprocessor, but you *will understand what these lines mean* and why errors happen here.

You won't write a compiler, but you'll learn how to work *with* one.

Structure of a C Program



Every C program must have a main function

Each function consists of a header followed by a basic block.

```
<return-type> fn-name (parameter-list)

{
declaration of variables executable
statements
}
```

<- Header

<- basic block

Main function

Every C program contains one function named main

The body of each function is enclosed by braces, { }

```
int main (void)
{
    declarations
    statements
}
```

Comments

Comments

1. Across lines

   begin with the characters /*

   end with the characters */

```
/*
    This is an example
*/
```

2. Same line

   // starts the comment which ends at the end of the line

```
int main (void) // Main function
{
    declarations
    statements
}
```

Return Statement

1. Sets the return value to the value of the expression
2. Returns to the caller / invoker

```
int main()              // header
{                       // beginning of basic block
                        // ...
    return 0;           // program ending successfully
}                       // end of basic block
```

Comments

Preprocessor Directives

C program line beginning with # that is processed by the compiler before translation begins.

- #include pulls another file into the source
  - `#include <stdio.h>` causes the contents of the named file, stdio.h, to be inserted where the # appears. File is commonly called a header file.
    - <>'s indicate that it is a compiler standard header file.
  - `#include "myfunctions.h"` causes the contents of myfunctions.h to be inserted
    - "'s indicate that it is a user file from current or specified directory

Constants and Variables

- A constant is a specific value and can not be changed
- A variable is a memory location that is assigned a name or an identifier
  - A variable is associated with a data type
  - Variables must be declared before the variable can be used

Variables

- An identifier is used to reference a memory location.
- Rules for selecting a valid identifier
  - must begin with an alphabetic character or underscore
  - may contain only letters, digits and underscore (no special characters)
  - case sensitive
  - cannot use keywords as identifiers

Keywords - Reserved words

Reserved word

- Word that has a specific meaning in C
- Cannot be used as an identifier (variable) name

Keywords in C Programming			
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Symbolic Constants

- Defined with a preprocessor directive
- Compiler replaces
  - each occurrence of the directive identifier with the constant value in all statements that follow the directive
- Example

```
#define    PI   3.141593
```

C Data Types

- Integers
  - **short**: 16 bits
  - **int**: 32 bits
  - **long**: 64 bits
- Floating-Point Values
  - **float**: 32 bits
  - **double**: 64 bits
  - **long double**: 64 or 128 bits
- Characters
  - **char**: 1 byte

These are the only guarantees C makes:

Data type	Minimum size
`char`	**1 byte** (always)
`short`	≥ 2 bytes
`int`	≥ `short`
`long`	≥ `int`
`long long`	≥ `long`
`float`	≥ 4 bytes
`double`	≥ `float`
`long double`	≥ `double`

Only char is fixed. Everything else is “at least”.

Assignment Statements

- Used to assign a value to a variable
- General Form:

```
identifier = expression;
```

- Example 1

```
double sum = 0;
```

- Example 2

```
int x;
x=5;
```

Assignment Statements

- Example 3

```
char ch;
ch = 'a';
```

- Example 4

```
int x, y, z;
x = y = 0;
z = 2;
```

- Example 5

```
y = x + z;
```

Arithmetic Operators

- Addition +
- Subtraction -
- Multiplication *
- Division /
- Modulus %
  - Modulus returns the remainder of a division between two integers
  - Example
    - 5 % 2 returns 1
    - 6 % 2 returns 0
    - 1 % 2 returns 1

Integer Division

- Division between two integers results in an integer.
- The result is truncated, not rounded
- Example:
  - 5/3 is equal to 1
  - 3/6 is equal to 0

Precedence of Operators

- Parentheses Inner most first
- Unary operators Right to left

  (+ -)
- Binary operators Left to right

  (* / %)
- Binary operators Left to right

  (+ -)

Precedence Example

- Find the average of three variables a, b and c

  Do not use: `a + b + c / 3`

  Use: `(a + b + c ) / 3`

- 11 / 2 / 2.0 / 2 performs 3 separate divisions.
  1. **11 / 2 → 5**
  2. **5 / 2.0 → 2.5**
  3. **2.5 / 2 → 1.25**

Increment and Decrement Operators

- Increment Operator ++
  - post increment x++;
  - pre increment ++x;
- Decrement Operator --
  - post decrement x--;
  - pre decrement --x;

Abbreviated Assignment Operator

- Operator Example Equivalent

+=	x += 2;	x = x + 2;
-=	x -= 2;	x = x - 2;
*=	x *= y;	x = x * y;
/=	x /= y;	x = x / y;
%=	x %= y;	x = x % y;

Standard Output

- **printf** function
  - prints information to the screen
  - requires one or more arguments
    - Req: control string with conversion specifiers
    - Opt: values that correspond to the specifiers in the control string
- Example

```
float angle = 45.5;
printf("Angle = %.2f degrees\n", angle);
```

- Output

```
Angle = 45.50 degrees
```

Printf() Reference

Type	Constant Examples	printf chars
char	'a', '\n'	%c
_Bool	0, 1	%i, %u
short int	—	%hi, %hx, %ho
unsigned short int	—	%hu, %hx, %ho
int	12, -97, 0xFFE0, 0177	%i, %x, %o
unsigned int	12u, 100U, 0XFFu	%u, %x, %o
long int	12L, -2001, 0xffffL	%li, %lx, %lo
unsigned long int	12UL, 100ul, 0xffeeUL	%lu, %lx, %lo
long long int	0xe5e5e5e5LL, 500ll	%lli, %llx, &llo
unsigned long long int	12ull, 0xffeeULL	%llu, %llx, %llo
float	12.34f, 3.1e-5f, 0x1.5p10,0x1P-1	%f, %e, %g, %a
double	12.34, 3.1e-5, 0x.1p3	%f, %e, %g, %a
long double	12.341, 3.1e-5l	%Lf, $Le, %Lg

Printf() – Return

A successful completion of printf returns the number of characters printed.

```
int num1 = 55;
int num2 = 30;
int sum = num1 + num2;
int printCount;
printCount = printf("%d + %d = %d\n", num1, num2, sum);
```

What is the value of printcount ?

Standard Input

- **scanf** function
  - inputs values from the keyboard
  - required arguments
    - control string with conversion specifiers
    - memory locations that correspond to the specifiers in the control string
- Example:

```
int distance;
char unit_length;
scanf("%d%c", &distance, &unit_length);
```

Standard Input

- scanf function
  - It is very important to use a specifier that is appropriate for the data type of the variable

Standard Input and Output

- Examples of specifiers
  - int – %d, %i
  - short – %hd, %hi
  - long – %ld, %li
  - char – %c
  - float – %f
  - double – %lf

Scanf()

```
scanf(format-string, address-list)
```

```
scanf("%d", &age);
```

%d – Tells scanf that the program is expecting an ASCII encoded integer number to be typed in, and that scanf should convert the string of ASCII characters to internal binary integer representation.

Address-list: List of memory addresses to hold the input values .

Scanf() cares about **where** the input should be stored; rather than **what** the input actually is..

Scanf() - Return

A successful completion of scanf() returns the number of input values read. Returns EOF if hits end-of-file reading one item.

Consequently, we could have

```
int dataCount;
dataCount = scanf("%d %d", &height, &weight);
```

- If scanf() is successful,
  the value in dataCount should be 2

- Spaces or new lines separate one value from another

Summary

- Basic C program structure
- Libraries
- Basic Function format
- Data types
- Keywords – Variables – Constants
- Printf()
- Scanf()
- Assignment Operators